

Lógica y Representación 1

Conceptos básicos de Python

Ingeniería de Sistemas

Universidad de Antioquia

Método de Polya - Resumen



1

Entender el problema

2

Diseñar un plan

3

Implementar el plan

4

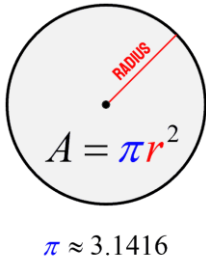
Revisar el plan

Resumiendo del proceso de diseño del algoritmos

Ejemplo:

Hacer un algoritmo que calcule el área de un círculo

1



2

Análisis:

Datos de entrada: radio

Datos de salida: área del círculo

Constantes:

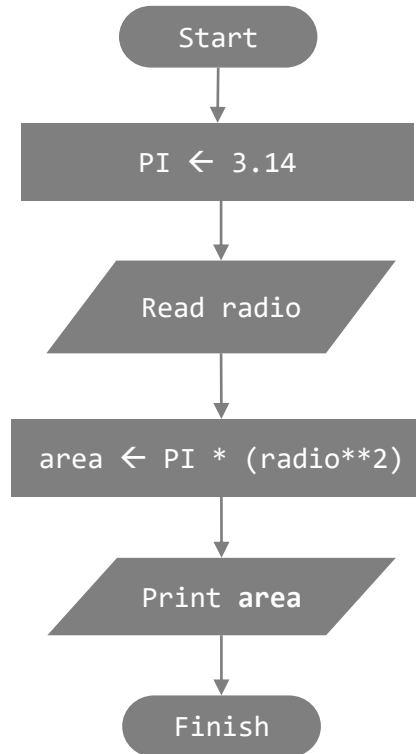
- **Numero PI:** $PI = 3.14$ (Real)

Definición de variables:

- **radio:** radio (Real)
- **area:** radio (Real)

¿Proceso?: Elevar el radio al cuadrado y multiplicarlo por π

3



Inicio

$PI = 3.14$

Leer(radio)

$area = PI * (radio**2)$

Escribir(area)

Fin

4

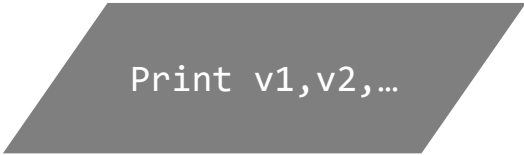
Entradas		Salidas	
radio		area	
10		314	
2.6		21.237	

↘radio	↗area
?	?
↘10	↗314

↘radio	↗area
?	?
↘2.6	↗21.226

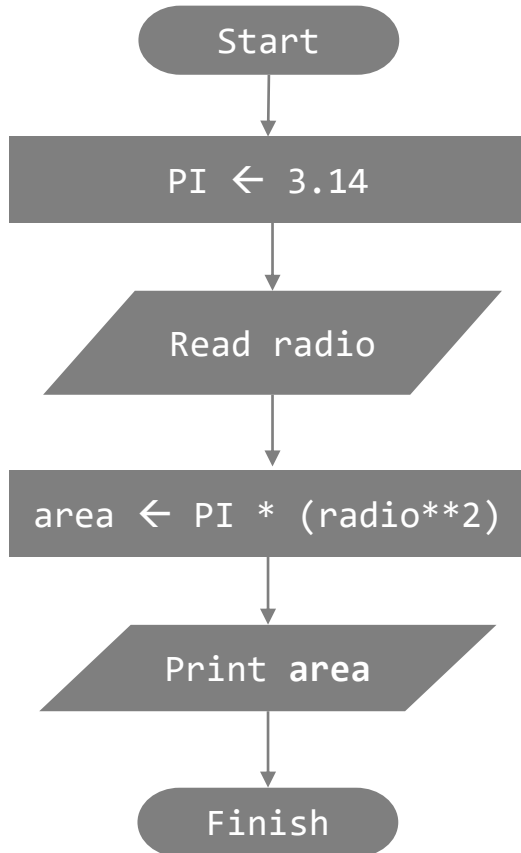
Pseudocodigo

La siguiente tabla muestra el resultado equivalente para operaciones secuenciales básicas entre el diagrama de bloques y el pseudocódigo (con las estructuras vistas hasta el momento).

Tipo	Diagrama de bloques	Pseudocodigo
Entrada 		Leer(v1,v2,...)
Salida 		Escribir(v1,v2,...)
Proceso 		$c = a + b$

Sobre los programas

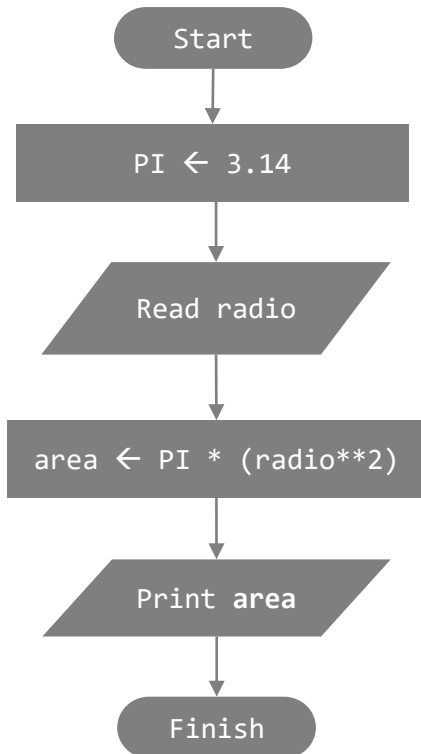
- Ya hemos logrado describir los pasos para plantear un algoritmo (algo natural para nosotros) de manera mas formal (diagrama de flujo y pseudocódigo), pero todavía no tenemos forma de ejecutarlo en una maquina.
- Necesitamos codificar el algoritmo en un lenguaje de programación para convertirlo en un programa ejecutable.



Programa en lenguaje Python

Problema:

Hacer un algoritmo que calcule el área de un círculo (Pseudocódigo forma 1)



Inicio

```
PI = 3.14
Leer(radio)
area = PI*(radio**2)
Escribir(area)
```

Fin

Inicializacion

```
PI = 3.14
```

Entrada de datos

```
radio = float(input("Digite el radio: "))
```

Calculo del area

```
area = PI * (radio**2)
```

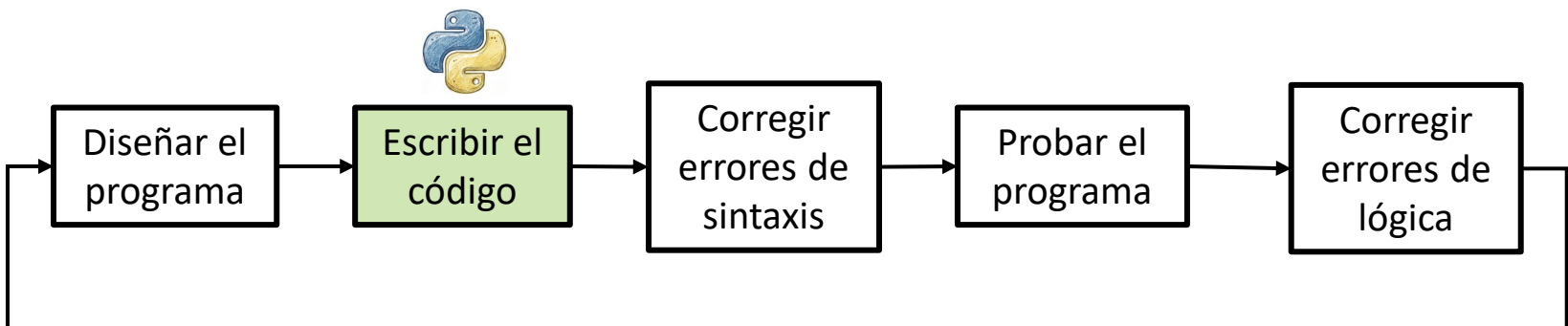
Despliegue del area

```
print("El área del círculo es:", area)
```

[link](#)

Diseño de programas

- Los programas deben diseñarse cuidadosamente antes de ser escritos. Durante el proceso de diseño, los programadores utilizan herramientas como pseudocódigo y diagramas de flujo para crear modelos de los programas.
- **El ciclo de desarrollo de un programa:**
 1. Diseñar el programa
 2. Escribir el código
 3. Corregir errores de sintaxis
 4. Probar el programa
 5. Corregir errores de lógica



Entrada, procesamiento y salida

Los programas de computadora realizan estos tres pasos:

1. Se recibe la **entrada**.
2. Se realiza un **proceso** sobre la entrada.
3. Se genera la **salida**



Inputs



Process



Outputs





Variables

- Las **variables** permiten el almacenamiento de datos en memoria.

Grupo	Pseudocódigo	Python	Explicación	Ejemplo
Numéricas	Entero	int	Números sin parte decimal, positivos o negativos	edad = 25
Numéricas	Real	float	Números con parte decimal	altura = 1.75
Lógicas	Lógico / Booleano	bool	Solo dos valores posibles: True o False	activo = True
Alfanuméricas	Carácter	str	Python no distingue carácter de cadena; se representa igual	inicial = "A"
Alfanuméricas	Cadena (Texto)	str	Secuencia de caracteres entre comillas (dobles o simples)	nombre = "Ana"

- En Python, carácter y cadena son el mismo tipo (str); no existe un tipo separado para un solo carácter.
- El tipo se asigna automáticamente al declarar la variable (no se declara explícitamente como en otros lenguajes).
- Las variables tipo cadena merecen un poco mas de atención. Por lo que serán analizadas en breve con un poco mas de detenimiento.



Variables

- Las **variables** permiten el almacenamiento de datos en memoria.

```
# Numéricas
Entero edad = 25
Real altura = 1.75

# Lógica
Logica es_estudiante = True

# Alfanuméricas
Texto inicial = "A"
Texto nombre = "Albert Einstein"
```

```
# Numéricas
edad = 25 # int
altura = 1.75 # float

# Lógica
es_estudiante = True # bool

# Alfanuméricas
inicial = "A" # str (carácter)
nombre = "Ana María" # str (cadena)

# Mostrando los valores y su tipo
print(nombre, type(nombre))
print(edad, type(edad))
print(altura, type(altura))
print(es_estudiante, type(es_estudiante))
```



Variables

- Las **variables** permiten el almacenamiento de datos en memoria.

Pseudocódigo

```
# Numéricas
Entero edad = 25
Real altura = 1.75

# Lógica
Logica es_estudiante = True

# Alfanuméricas
Texto inicial = "A"
Texto nombre = "Albert Einstein"
```

Código python

```
# Numéricas
edad = 25                # int
altura = 1.75            # float

# Lógica
es_estudiante = True    # bool

# Alfanuméricas
inicial = "A"            # str (carácter)
nombre = "Albert Einstein" # str (cadena)
```

[link](#)

Memoria

edad	25
altura	1.75
es_estudiante	True
inicial	"A"
nombre	"Albert Einstein"
...	





Variables

- Las **variables** permiten el almacenamiento de datos en memoria.

Código python

```
# Numéricas
edad = 25                # int
altura = 1.75            # float

# Lógica
es_estudiante = True    # bool

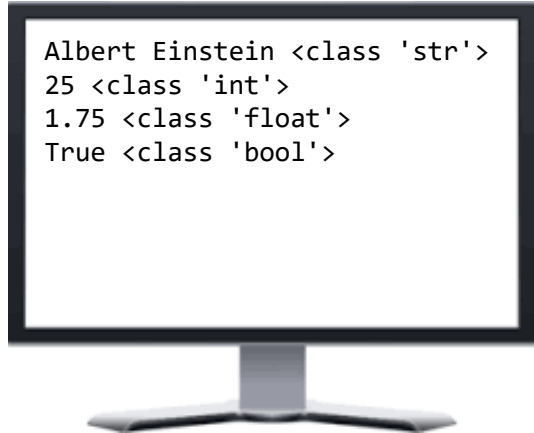
# Alfanuméricas
inicial = "A"            # str (carácter)
nombre = "Albert Einstein" # str (cadena)

# Mostrando los valores y su tipo
print(nombre, type(nombre))
print(edad, type(edad))
print(altura, type(altura))
print(es_estudiante, type(es_estudiante))
```

[link](#)

Memoria

edad	25
altura	1.75
es_estudiante	True
inicial	"A"
nombre	"Albert Einstein"
...	



```
Albert Einstein <class 'str'>
25 <class 'int'>
1.75 <class 'float'>
True <class 'bool'>
```

Prueba de escritorio

Dado las siguientes operaciones aritméticas, obtenga los valores finales asociados a las variables.

```
cantidad = 10
precio = 2.5
total = cantidad * precio
descuento = 5.0
total = total - descuento
cantidad = cantidad + 3
total = total + cantidad
promedio = total / cantidad
```

[link](#)

cantidad	precio	total	Descuento
?	?	?	?



Sobre las cadenas en python

Creación de cadenas

- En Python, las cadenas se pueden escribir con comillas simples `' '` o dobles `" "` — no hay diferencia funcional entre ellas.
- Se usan comillas triples `''' '''` o `""" """` cuando el texto ocupa varias líneas.
- Si el texto contiene una comilla, se usa el otro tipo de comilla para evitar conflicto.

```
# Comillas simples y dobles
(equivalentes)
nombre = 'Ana'
ciudad = "Medellín"

# Evitando conflicto de comillas
frase = "Ella dijo: 'Hola'"

# Cadena multilínea
mensaje = """Bienvenido al curso.
Hoy hablaremos de cadenas."""

print(frase)
print(mensaje)
```

[link](#)

Print output (drag lower right corner to resize)

```
Ella dijo: 'Hola'
Bienvenido al curso.
Hoy hablaremos de cadenas.
```

Frames

Objects

Global frame

nombre	"Ana"
ciudad	"Medellín"
frase	"Ella dijo: 'Hola'"
mensaje	"Bienvenido al curso. Hoy hablaremos de cadenas."



Sobre las cadenas en python

Concatenación y repetición

- El operador **+** une (concatena) dos o más cadenas en una sola. (todas las variables concatenadas deben ser **str**)
- El operador ***** repite una cadena un número determinado de veces. (requiere una cadena y un entero).

```
# Concatenación
nombre = "Ruben"
apellido = "Aguirre"
nombre_completo = nombre + " " + apellido
print(nombre_completo)    # Ruben Aguirre

# Repetición
linea = "-" * 20
print(linea)
# -----

separador = "=" * 10
print(separador + " Fin " + separador)
# ===== Fin =====
```

[link](#)

Print output (drag lower right corner to resize)

```
Ruben Aguirre
-----
===== Fin =====
```

Global frame

nombre	"Ruben"
apellido	"Aguirre"
nombre_completo	"Ruben Aguirre"
linea	"-----"
separador	"=====



Sobre las cadenas en python

Caracteres de escape

- Los caracteres de escape permiten incluir símbolos especiales dentro de una cadena, usando una barra invertida \.
- Se usan cuando el símbolo no se puede escribir directamente (como un salto de línea) o genera conflicto (como una comilla).

Código	Efecto	Código	Efecto
<code>\n</code>	Salto de línea	<code>\\</code>	Barra invertida
<code>\t</code>	Tabulación	<code>\r</code>	Retorno de carro
<code>\"</code>	Comilla doble	<code>\b</code>	Retroceso (backspace)
<code>\'</code>	Comilla simple	<code>\a</code>	Alerta (sonido)
<code>\f</code>	Salto de página	<code>\v</code>	Tabulación vertical
<code>\0</code>	Carácter nulo	<code>\ooo</code>	Notación octal (ej. <code>\101</code> → A)
<code>\xhh</code>	Notación hexadecimal (ej. <code>\x41</code> → A)	<code>\N{name}</code>	Carácter Unicode por nombre
<code>\uxxxx</code>	Unicode 16 bits	<code>\Uxxxxxxxx</code>	Unicode 32 bits



Sobre las cadenas en python

Concatenación y repetición

- La barra invertida `\` al final de una línea (fuera de una cadena, o justo antes del salto de línea física del archivo) indica que la instrucción continúa en la siguiente línea.
- Es distinto al uso dentro de cadenas de texto (donde `\` inicia un carácter de escape como `\n` o `\t`).
- Se usa por legibilidad, cuando una línea de código es demasiado larga.

```
print("Científico:\tJohn von Neumann")
print("Área:\t\tMatemáticas y Ciencias de la Computación")
print("Frase célebre:\n"En matemáticas no se entienden las cosas, \
      uno simplemente se acostumbra a ellas.\")
print("Nota:\tPionero de la arquitectura \"von Neumann\", \
      base de las computadoras modernas.")
```

[link](#)

Print output

```
Científico:      John von Neumann
Área:           Matemáticas y Ciencias de la Computación
Frase célebre:
"En matemáticas no se entienden las cosas, uno           simplemente se acostumbra a ellas."
Nota:   Pionero de la arquitectura "von Neumann",       base de las computadoras modernas.
```



Sobre las cadenas en python

Indexado

- Cada carácter de una cadena tiene una **posición (índice)**, empezando en **0**.
- Se accede a un carácter con **cadena[posición]**.
- Los índices negativos cuentan desde el final: **-1** es el último carácter.
- El espacio en blanco también es un carácter y cuenta para la indexación.
- **len(cadena)** devuelve la cantidad total de caracteres.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
U	n	i	v	e	r	s	i	d	a	d		d	e		A	n	t	i	o	q	u	i	a
-24	-23	-22	-21	-20	-19	-18	-17	-16	-15	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

```
universidad = "Universidad de Antioquia"
```

```
print(universidad[0])      # U   (primer carácter)
print(universidad[11])     # espacio (entre "Universidad" y "de")
print(universidad[15])     # A   (inicio de "Antioquia")
print(universidad[-1])     # a   (último carácter)
print(universidad[-9])     # A   (mismo carácter, contado desde el final)
print(len(universidad))    # 24  (cantidad total de caracteres, incluye espacios)
```

[link](#)



Sobre las cadenas en python

Slicing (rebanado)

- El **slicing** permite extraer una **porción** (subcadena) usando **cadena[inicio:fin]**.
- El carácter en la posición **inicio** **sí se incluye**; el de la posición **fin** **no se incluye**.
- Si se omite inicio, empieza desde el principio (**0**).
- Si se omite fin, llega hasta el final de la cadena.
- Se puede agregar un tercer valor **[inicio:fin:paso]** para saltar caracteres.

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23
U	n	i	v	e	r	s	i	d	a	d		d	e		A	n	t	i	o	q	u	i	a
-24	-23	-22	-21	-20	-19	-18	-17	-16	-15	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

```
universidad = "Universidad de Antioquia"
```

```
print(universidad[0:11])      # Universidad
print(universidad[12:14])    # de
print(universidad[15:])       # Antioquia
print(universidad[:11])       # Universidad
print(universidad[-9:])       # Antioquia (usando índice negativo)
print(universidad[::-2])      # Uiestddatoqi (un carácter sí, uno no)
```



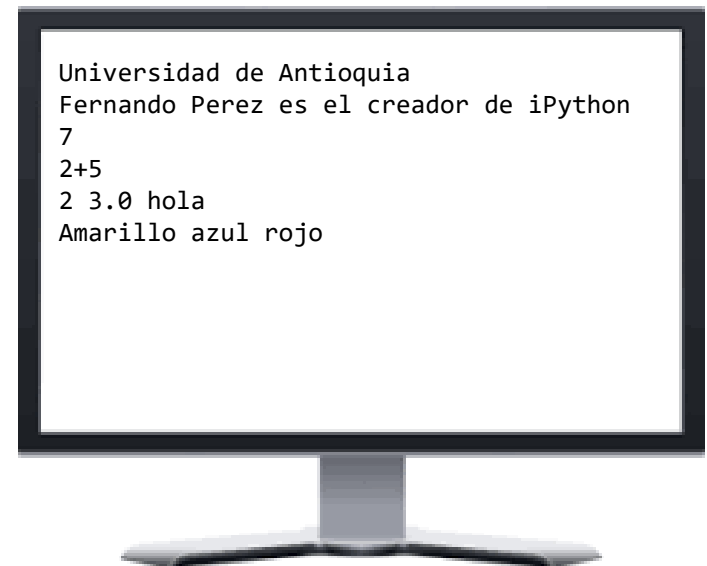
Mostrando la salida con la instrucción `print`

Instrucción `print`

- La salida en pantalla se realiza usando la función `print`.
- Muestra en pantalla texto, números o el resultado de una operación.
- Todo lo que va dentro de los paréntesis se muestra tal cual.
- El texto (cadenas de caracteres) se escribe entre comillas `" "` o `' '`.
- Se pueden imprimir varios valores separados por comas.
- Cada llamado a `print()` inicia en una nueva línea.

```
print("Universidad de Antioquia")  
print('Fernando Perez creo iPython')  
print(2+5)  
print('2+5')  
print(2,3.0, "hola")  
print("Amarillo", "azul", "rojo")
```

[link](#)





Mostrando la salida con la instrucción `print`

Sintaxis de `print`

```
print(valor1, valor2, ..., sep = ' ', end = '\n')
```

- **Donde:**
 - **valor1, valor2, ...:** Elementos a mostrar
 - **sep:** separador entre valores (por defecto un espacio)
 - **end:** qué se agrega al final (por defecto salto de línea)

Código	Salida
<pre>print("Today is Monday.") print("I like string beans.")</pre>	Today <code>is</code> Monday. I like string beans.
<pre>print("Today", "is", "Monday") print("Today", "is", "Monday", sep="...")</pre>	Today <code>is</code> Monday Today... <code>is</code> ...Monday
<pre>print("Today is Monday, ", end="") print("I like string beans.")</pre>	Today <code>is</code> Monday, I like string beans.
<pre>print("Today", "is", "Monday", sep="? ", end="!!!") print("I like string beans.")</pre>	Today? is? Monday!!!I like string beans.





Mostrando la salida con la instrucción `print`

Imprimiendo variables de diferentes tipos

- `print()` puede recibir variables de cualquier tipo: `int`, `float`, `str`, `bool`.
- Python convierte automáticamente cada valor a texto para mostrarlo en pantalla.
- Se pueden combinar varios tipos en un mismo `print()`, separados por comas.
- Para concatenar con `+`, todos los valores deben ser `str` — los números se deben convertir con `str()`.

```
universidad = "Universidad de Antioquia"
anio_fundacion = 1803
promedio = 4.35
activa = True

# Varios tipos separados por comas
print(universidad, anio_fundacion, promedio, activa)

# El mismo mensaje, usando comas (sin necesidad de convertir tipos)
print("La", universidad, "fue fundada en", anio_fundacion)

# El mismo mensaje, usando concatenación (requiere convertir a str)
print("La " + universidad + " fue fundada en " + str(anio_fundacion))
```

Print output :

```
Universidad de Antioquia 1803 4.35 True
La Universidad de Antioquia fue fundada en 1803
La Universidad de Antioquia fue fundada en 1803
```

Global frame

universidad	"Universidad de Antioquia"
anio_fundacion	1803
promedio	4.35
activa	True

[link](#)



Mostrando la salida con la instrucción `print`

Salida formateada con `fstrings`

- Las **f-strings** permiten insertar variables directamente dentro de un texto, usando `f"..."` y llaves `{ }`.
- Son más legibles que concatenar con `+` o separar con comas.
- Dentro de las llaves se pueden usar variables e incluso operaciones.
- Se puede dar formato a números dentro de las llaves (ejemplo: `{promedio:.2f}` para 2 decimales).

```
universidad = "Universidad de Antioquia"
anio_fundacion = 1803
promedio = 4.35
antiguedad = 2026 - anio_fundacion

print(f"La {universidad} fue fundada en {anio_fundacion}.")
print(f"Tiene aproximadamente {antiguedad} años de historia.")
print(f"El promedio del estudiante es {promedio:.2f}")
```

[link](#)

Print output

```
La Universidad de Antioquia fue fundada en 1803.
Tiene aproximadamente 223 años de historia.
El promedio del estudiante es 4.35
```

Frames

Global frame

universidad	"Universidad de Antioquia"
anio_fundacion	1803
promedio	4.35
antiguedad	223



Mostrando la salida con la instrucción **print**

Tabla resumen de formateo con fstrings

- Dentro de **{ }** no solo van variables: se puede escribir cualquier expresión válida de Python (operaciones, formato, métodos, etc.).

Código	Qué hace {}	Salida
<pre>programa = "Ingeniería de Sistemas" print(f"Programa: {programa}")</pre>	Inserta el valor de una variable	Programa: Ingeniería de Sistemas
<pre>nota1 = 4.2 nota2 = 3.8 print(f"Promedio: {(nota1 + nota2) / 2}")</pre>	Calcula una operación entre variables	Promedio: 4.0
<pre>promedio = 4.3567 print(f"Promedio: {promedio:.2f}")</pre>	Redondea el valor a 2 decimales	Promedio: 4.36
<pre>nombre = "ana" print(f"Bienvenida, {nombre.upper()}")</pre>	Llama a un método sobre la variable	Bienvenida, ANA



Entrada de datos usando `input`

Instrucción `input`

- La función `input()` permite **recibir datos** desde el teclado mientras el programa se ejecuta.
- El programa se detiene y espera a que el usuario escriba algo y presione **Enter**.
- Dentro de los paréntesis se puede escribir un **mensaje (prompt)** que guía al usuario sobre qué debe ingresar.
- `input()` siempre devuelve el valor como **texto (str)**, sin importar si el usuario escribió un número.

```
nombre = input("¿Cómo te llamas? ")  
print("Hola,", nombre)
```

[link](#)

Print output

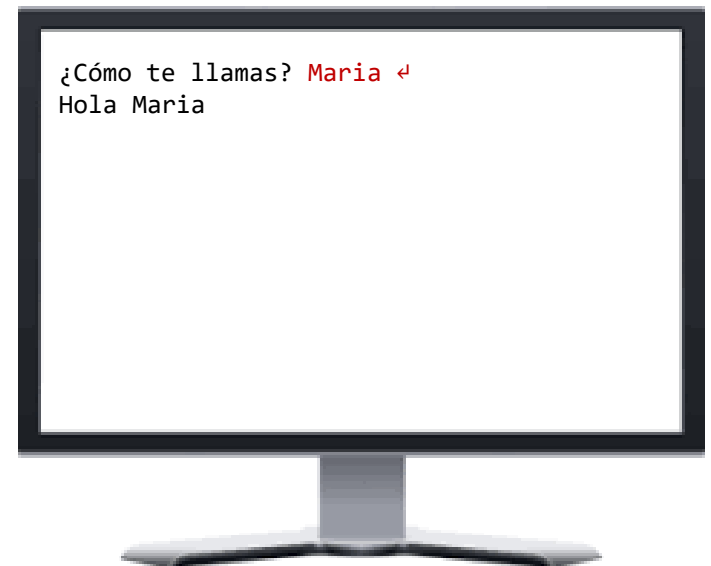
```
¿Cómo te llamas? Maria  
Hola, Maria
```

Frames

Objects

Global frame

nombre	"Maria"
--------	---------





Entrada de datos usando `input`

Sintaxis de `input`

```
variable = input("mensaje")
```

- El **mensaje (prompt)** es opcional, pero es una buena práctica incluirlo siempre para que el usuario sepa qué debe escribir.
- Si no se asigna el resultado a una variable, el valor ingresado se pierde — no queda almacenado en memoria.

Código	Salida (interacción)
<code>input("Escribe algo: ")</code>	Escribe algo: Hola ↵ <i>(el valor no se guarda en ninguna variable)</i>
<code>dato = input("Escribe algo: ") print(dato)</code>	Escribe algo: Hola ↵ Hola
<code>edad = input("¿Cuántos años tienes? ") print("Tipo:", type(edad))</code>	¿Cuántos años tienes? 20 ↵ Tipo: <class 'str'>





Entrada de datos usando `input`

Conversión de tipos con `input`

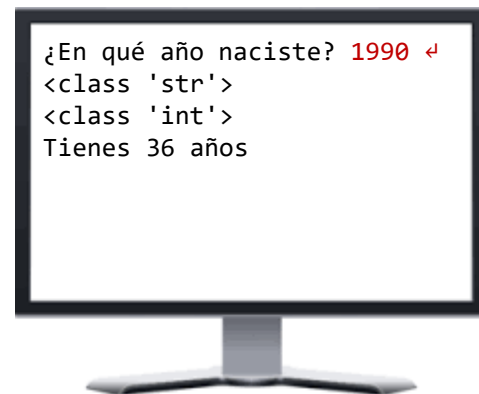
- `input()` siempre devuelve texto (**str**), aunque el usuario escriba un número.
- Para poder operar con el valor ingresado (sumar, restar, comparar), hay que convertirlo explícitamente.
- `int()` convierte a número entero; `float()` convierte a número decimal.
- Si se intenta operar sin convertir, Python genera un error de tipo (**TypeError**).

```
anio_nacimiento = input("¿En qué año naciste? ")
print(type(anio_nacimiento))          # <class 'str'>

anio_nacimiento = int(anio_nacimiento)
print(type(anio_nacimiento))          # <class 'int'>

edad = 2026 - anio_nacimiento
print("Tienes", edad, "años")
```

[link](#)



```
anio_nacimiento = input("¿En qué año naciste? ")
edad = 2026 - anio_nacimiento      # TypeError
```

[link](#)





Entrada de datos usando `input`

Combinando `input` con `print` y f-strings

- Con `input()`, `int()/float()` y **f-strings**, ya se puede construir un programa completo que siga el modelo **entrada → proceso → salida**.
- El resultado se puede mostrar de forma clara y legible usando **f"..."**.

```
nombre = input("¿Cómo te llamas? ")
anio_nacimiento = int(input("¿En qué año naciste? "))

edad = 2026 - anio_nacimiento

print(f"Hola, {nombre}. Tienes {edad} años.")
```

[link](#)

Print output

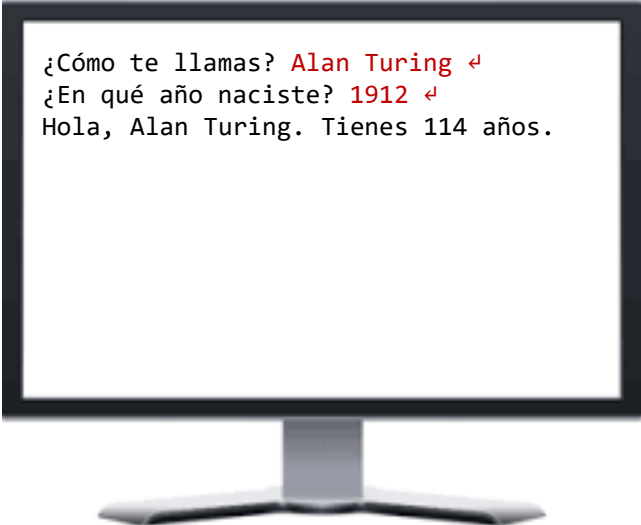
```
¿Cómo te llamas? Alan Turing
¿En qué año naciste? 1912
Hola, Alan Turing. Tienes 114 años.
```

Frames

Objects

Global frame

nombre	"Alan Turing"
anio_nacimiento	1912
edad	114



```
¿Cómo te llamas? Alan Turing ↵
¿En qué año naciste? 1912 ↵
Hola, Alan Turing. Tienes 114 años.
```

Comentarios

- Los **comentarios** son sentencias que permiten explicar las líneas de código de un programa.
- Los comentarios siempre son ignorados por el compilador o interprete del lenguaje.
- Dependiendo del lenguaje, pueden existir dos tipos de comentarios:

- **Comentarios de una línea:** son aquellos que inician con el símbolo **#**. Estos comentarios se utilizan para explicar de manera corta una instrucción.

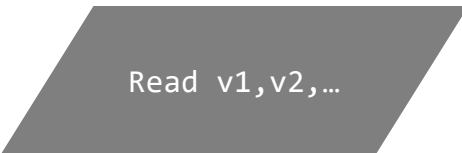
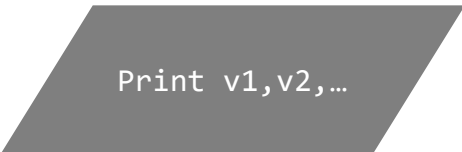
```
# Esto es un comentario de una línea
```

- **Comentarios varias líneas:** son aquellos que inician con los símbolos **" " "** y terminan con los símbolos **" " "**. Se usan para describir o explicar el funcionamiento de un bloque de código.

```
"""  
Este es un comentario  
de varias  
líneas"""
```

Tabla resumen

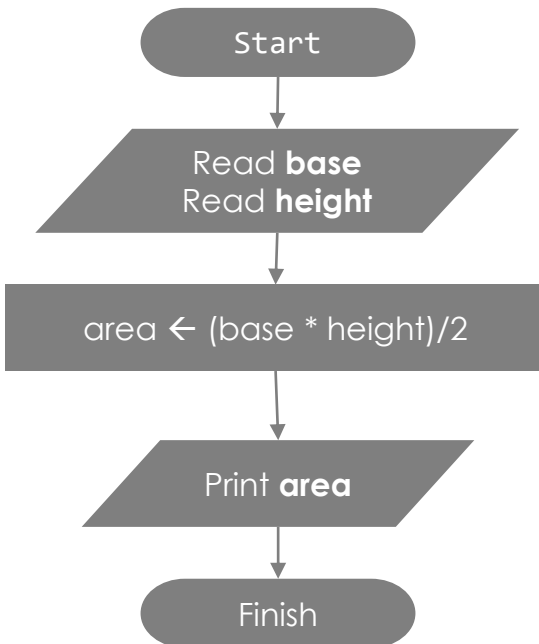
La siguiente tabla muestra la representación equivalente de las piezas esenciales para construir código secuencial

Tipo	Diagrama de bloques	Pseudocodigo	Python
Entrada 		Leer(v1,v2,...)	<pre>v1 = input(...) v2 = input(...)</pre>
Salida 		Escribir(v1,v2,...)	<pre>print(v1,v2,...)</pre>
Proceso 		$c = a + b$	$c = a + b$

En Python, `input()` siempre devuelve texto (`str`); si el valor se va a usar en una operación numérica, debe convertirse con `int()` o `float()` (por ejemplo: `v1 = float(input(...))`).

Codificación de algoritmos en Python

Ejemplo 1: Realice un programa que permita calcular al área de un triángulo



Inicio

```
Leer(base)
Leer(height)
area = (base*height)/2
Escribir(area)
```

Fin

```
# Entrada de datos
base = float(input("Ingrese la base del triangulo: "))
height = float(input("Ingrese la altura del triangulo: "))

# Calculo del area
area = (base * height)/2

# Despligue del area
print("El area del triangulo es:", area)
```



[link](#)

Codificación de algoritmos en Python

Ejemplo 1: Realice un programa que permita calcular al área de un triángulo

```
# Entrada de datos
base = float(input("Ingrese la base del triángulo: "))
height = float(input("Ingrese la altura del triángulo: "))

# Calculo del area
area = (base * height)/2

# Despliega del area
print("El area del triángulo es:", area)
```



[link](#)

Entradas		Salidas (esperadas)
base	Height	area
4	3	6
3	5	7.5

```
Terminal 1/A x
In [6]: runfile('D:/UdeA/2026-2/repos/logica1-20262/clases/3/ejemplo1-area.py',
wdir='D:/UdeA/2026-2/repos/logica1-20262/clases/3')
Ingrese la base del triángulo: 4
Ingrese la altura del triángulo: 6
El area del triángulo es: 12.0

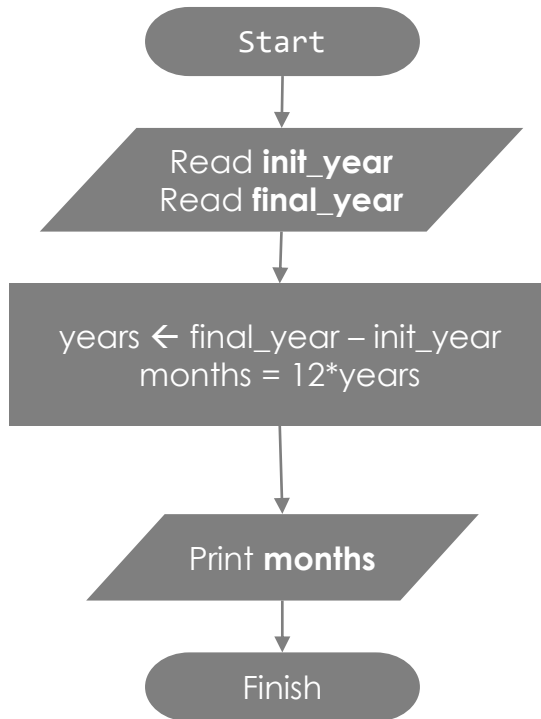
In [7]: runfile('D:/UdeA/2026-2/repos/logica1-20262/clases/3/ejemplo1-area.py',
wdir='D:/UdeA/2026-2/repos/logica1-20262/clases/3')
Ingrese la base del triángulo: 3
Ingrese la altura del triángulo: 5
El area del triángulo es: 7.5

In [8]: |
```

Terminal de IPython Historial

Codificación de algoritmos en Python

Ejemplo 2: Se requiere diseñar un algoritmo que **calcule el número de meses que hay entre los años A y B.**



Inicio

```
Leer(init_year)
Leer(final_year)
years = init_year, final_year
months = 12*years
Escribir(area)
```

Fin

```
# Entrada de datos
init_year = int(input("Ingrese el año inicial: "))
final_year = int(input("Ingrese el año final: "))
```

```
# Calculo del area
years = final_year - init_year
months = years * 12
```

```
# Despliegue de los meses
print(f"El número de meses entre los años {init_year} y {final_year} es: {months}")
```



Codificación de algoritmos en Python

Ejemplo 2: Se requiere diseñar un algoritmo que **calcule el número de meses que hay entre los años A y B.**

```
# Entrada de datos
init_year = int(input("Ingrese el año inicial: "))
final_year = int(input("Ingrese el año final: "))

# Calculo del area
years = final_year - init_year
months = years * 12

# Despligue de los meses
print(f"El número de meses entre los años {init_year} y {final_year} es: {months}")
```



[link](#)

Entradas		Salidas (esperadas)
init_year	final_year	months
2	12	120
4	10	72

```
Terminal 1/A x
In [8]: runfile('D:/UdeA/2026-2/repos/logical-20262/clases/3/ejemplo2-meses.py',
wdir='D:/UdeA/2026-2/repos/logical-20262/clases/3')
Ingrese el año inicial: 2
Ingrese el año final: 12
El número de meses entre los años 2 y 12 es: 120

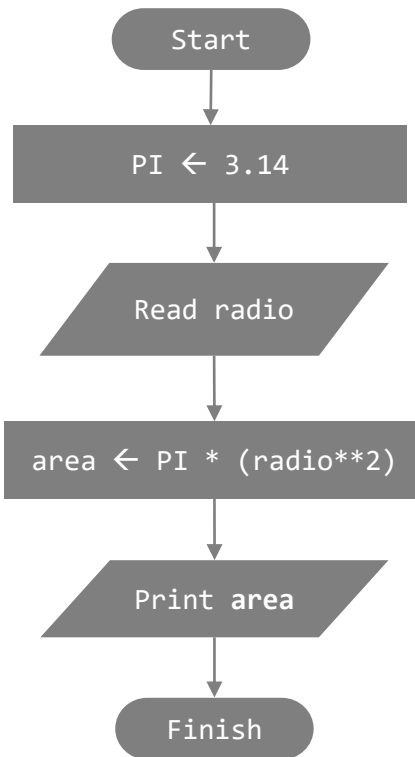
In [9]: runfile('D:/UdeA/2026-2/repos/logical-20262/clases/3/ejemplo2-meses.py',
wdir='D:/UdeA/2026-2/repos/logical-20262/clases/3')
Ingrese el año inicial: 4
Ingrese el año final: 10
El número de meses entre los años 4 y 10 es: 72

In [10]:
```

Terminal de IPython Historial

Codificación de algoritmos en Python

Ejemplo 3: Hacer un algoritmo que calcule el área de un círculo



Inicio

```
PI = 3.14
Leer(radius)
area = PI*(radio**2)
Escribir(area)
```

Fin

Inicialización de variables

```
PI = 3.14
```

Entrada de datos

```
radio = float(input("Ingresa el radio del círculo: "))
```

Calculo del area

```
area = PI * radio ** 2
```

Despliegue del area

```
print(f"El area del círculo de radio {radio} es: {area}")
```

[link](#)



Codificación de algoritmos en Python

Ejemplo 3: Hacer un algoritmo que calcule el área de un círculo

```
# Inicialización de variables
PI = 3.14

# Entrada de datos
radio = float(input("Ingrese el radio del círculo: "))

# Calculo del area
area = PI * radio ** 2

# Despligue del area
print(f"El area del círculo de radio {radio} es: {area}")
```



[link](#)

Entradas		Salidas	
radio		area	
10		314	
2.6		21.237	

```
Terminal 1/A x
In [10]: runfile('D:/UdeA/2026-2/repos/logica1-20262/clases/3/ejemplo3-circulo.py',
wdir='D:/UdeA/2026-2/repos/logica1-20262/clases/3')
Ingrese el radio del círculo: 10
El area del círculo de radio 10.0 es: 314.0

In [11]: runfile('D:/UdeA/2026-2/repos/logica1-20262/clases/3/ejemplo3-circulo.py',
wdir='D:/UdeA/2026-2/repos/logica1-20262/clases/3')
Ingrese el radio del círculo: 2.6
El area del círculo de radio 2.6 es: 21.2264

In [12]:
```

Terminal de IPython Historial

Ejercicios de repaso

Después de haber realizado en pseudocódigo o diagrama de flujo la representación de los algoritmos que se muestran a continuación, realice la codificación en Python y pruebe su funcionamiento.

1. Hacer un programa que calcule el área y el perímetro de un círculo dado su radio.
2. Hacer un programa que solicite el nombre, la identificación, el número de horas trabajadas y el valor por hora y realice el cálculo del salario neto teniendo en cuenta que se cobra un impuesto del 10% por salud y un 5% debido a pensiones.
3. Hacer un programa que pida el nombre y la edad (en años) y retorne un mensaje con el nombre y la edad en días.
4. Hacer un programa que solicite los 2 catetos de un triángulo rectángulo y calcule su hipotenusa.
5. Diseñar un algoritmo que lea dos valores reales y nos muestre los resultados de sumar, restar, dividir y multiplicar dichos números.
6. Si una persona corta sus uñas en promedio una vez por cada 1.5 semanas, dada la edad de una persona calcular cuantas veces ha cortado sus uñas en toda su vida. Adicionalmente, si en cada corte en promedio obtiene 0.5 gramos del material de la uñas cuantos kilos lleva en su existencia.

Referencias

1. <https://ellibrodepython.com/>
2. <https://www.w3resource.com/index.php>
3. <https://realpython.com/>
4. <https://curriculumresources.edu.gh/wp-content/uploads/2025/01/>
5. <https://python-course.eu/>
6. <https://realpython.com/cheatsheets/python/>

Eso es todo amigos

